7.ROS2 节点

1.节点简介

在通信时，不论采用何种方式，通信对象的构建都依赖于节点（Node），在 ROS2 中，一般情况下每个节点都对应某一单一的功能模块（例如：雷达驱动节点可能负责发布雷达消息，摄像头驱动节点可能负责发布图像消息）。一个完整的机器人系统可能由许多协同工作的节点组成，ROS2 中的单个可执行文件（C++程序或 python 程序）可以包含一个或多个节点。

2.创建节点

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/hello_world_demo/hello_world_demo**”，回车，切换到“**hello_world_demo**”功能包的路径下。

```
ubuntu@ubuntu:~/ros2_ws/src/hello_world_demo$ cd ros2_ws/src/hello_world_demo/hello_world_demo
```

2） 输入指令“**gedit hello_world.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/hello_world_demo/hello_world_demo$ gedit hello_world.py
```

```
import rclpy

from rclpy.node import Node

import time



class HelloWorldNode(Node):

    def __init__(self):

        # 调用基类 Node 的构造函数，设置节点名称
```

```
        super().__init__('hello_world_demo')


    def run(self):
        # 在 ROS2 系统正常运行的情况下执行循环
        while rclpy.ok():
            # 打印 "Hello World" 到节点的日志
            self.get_logger().info('Hello World')
            # 休眠 0.5 秒，控制循环时间
            time.sleep(0.5)


def main(args=None):
    # 初始化 ROS2 Python 接口
    rclpy.init(args=args)
    # 创建 HelloWorldNode 实例
    node = HelloWorldNode()


    try:
        # 运行节点的主循环
        node.run()
    except KeyboardInterrupt:
        pass
    finally:
```

```
        # 销毁节点对象
        node.destroy_node()
        # 关闭 ROS2 Python 接口
        rclpy.shutdown()


if __name__ == '__main__':
    # 执行主函数
    main()
```

3） 输入指令“**chmod +x hello_world.py**”，并按下回车，为保存的 hello_world.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/hello_world_demo/hello_world_demo$ chmod +x hello_world.py
```

3.编译运行

1） 赋予可执行权限后，输入“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~/ros2_ws/src/hello_world_demo/hello_world_demo$ cd ~/ros2_ws/
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

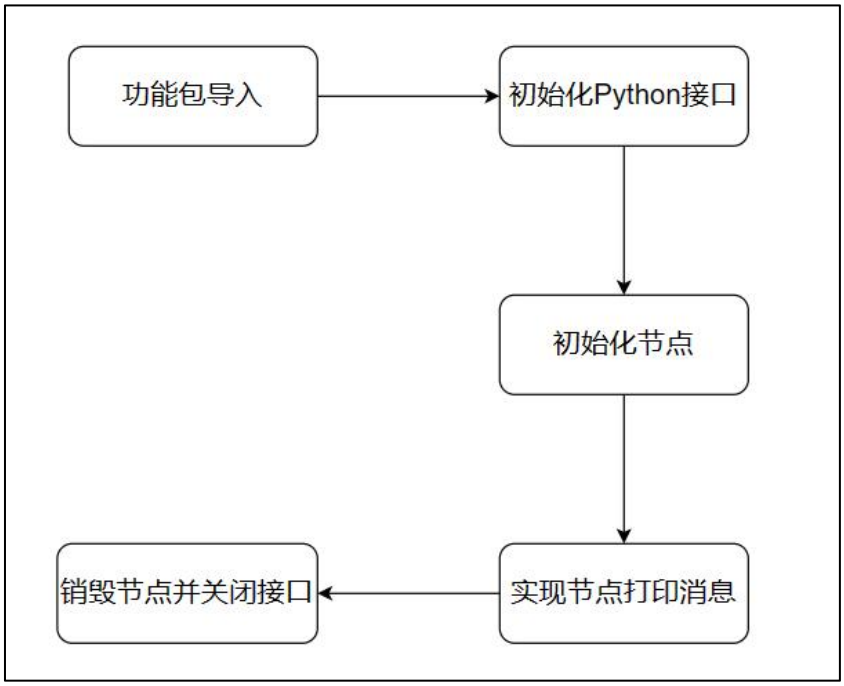
```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 run hello_world_demo hello_world**”，并按下回车，启动 hello_world 节点。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run hello_world_demo hello_world
[INFO] [1709024495.479936514] [hello_world_demo]: Hello World
[INFO] [1709024495.981809618] [hello_world_demo]: Hello World
[INFO] [1709024496.484232230] [hello_world_demo]: Hello World
[INFO] [1709024496.986541816] [hello_world_demo]: Hello World
[INFO] [1709024497.488489640] [hello_world_demo]: Hello World
```

4.程序分析

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个名为 HelloWorldNode 的节点，并在节点的主循环中以 0.5 秒的间隔打印"Hello World"到节点的日志中。在程序运行时，首先会初始化 ROS2 Python 接口，然后创建节点实例并运行其主循环。当程序被中断时，将销毁节点对象并关闭 ROS2 Python 接口。

◆ 主函数

```
def main(args=None):
    # 初始化 ROS2 Python 接口
    rclpy.init(args=args)
    # 创建 HelloWorldNode 实例
    node = HelloWorldNode()

    try:
        # 运行节点的主循环
        node.run()
    except KeyboardInterrupt:
        pass
    finally:
        # 销毁节点对象
        node.destroy_node()
        # 关闭 ROS2 Python 接口
        rclpy.shutdown()
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 HelloWorldN

ode，然后执行 node.run()函数。

◆ HelloWorldNode 类

```
class HelloWorldNode(Node):
    def __init__(self):
        # 调用基类 Node 的构造函数，设置节点名称
        super().__init__('hello_world_demo')

    def run(self):
        # 在 ROS2 系统正常运行的情况下执行循环
        while rclpy.ok():
            # 打印 "Hello World" 到节点的日志
            self.get_logger().info('Hello World')
            # 休眠 0.5 秒，控制循环时间
            time.sleep(0.5)
```

首先创建了一个名为 HelloWorldNode 的节点，run()函数里面在在节点的主循环中以 0.5 秒的间隔打印"Hello World"到节点的日志中。